NAME: S.KARTHIKEYAN **3122 22 5001 056**
CSE B

UCS 2312 Data Structures Lab

Assignment 10: Implementation of Shortest Path Finding algorithm

Date of Assignment: 18.11.2023

The cityADT contains the number of cities and the connectivity information between the cities (adjacency matrix). Write the following methods. [CO2, K3]

- void create(cityADT *C) – will represent the graph using adjacency matrix
- void disp(cityADT *C) – Display the graph
- void Dijkstra(cityADT *C) – Displays the intermediate and final tables
- char * displayPath(cityADT *C, source, destination) – Find the path of the intermediate cities between the source and destination cities along with the cost

CODE:

Djadt.h

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

struct table
{
    int known;
    int dist;int path;
};

struct graph
{
    int edges;int vertices;
    int arr[100][100];
    struct table vertex[20];
};

struct edgepair
{
    int start;int end;
    int weight;
};

void init(struct graph *g,int n,int e)
{
```

NAME: S.KARTHIKEYAN 3122 22 5001 056
CSE B

```
    g->vertices=n;
    g->edges=e;
    for (int i=0;i<=g->vertices;i++)
    {   for (int j=0;j<=g->vertices;j++)
        {
            g->arr[i][j]=0;
        }
    }
}

void create(struct graph *g,struct edgepair *err[])
{
    for (int i=0;i<g->edges;i++)
    {
        g->arr[err[i]->start-1][err[i]->end-1]=err[i]->weight;
        //g->arr[err[i]->end][err[i]->start]=1; undirected graph

    }
    printf("done create\n");


}

void display(struct graph *g)
{
    for (int i=0;i<g->vertices;i++)
    {   for (int j=0;j<g->vertices;j++)
        {   printf("%d ",g->arr[i][j]);}
        printf("\n");

    }
}

void inti(int start,struct graph *g)
{
    start=start-1;
    for (int i =0;i<g->vertices;i++)
    {
        g->vertex[i].known=0;
        g->vertex[i].dist=999;
        g->vertex[i].path=-1;
        g->vertex[start].dist=0;
    }
}
```

NAME: S.KARTHIKEYAN 3122 22 5001 056
CSE B

```
int numofunknown(struct graph *g)
{
    int c=0;
    for (int i=0;i<g->vertices;i++)
    {
        if (g->vertex[i].known==0)
            c++;
    }
    return c;
}

int smallestvertex(struct graph *g)
{
    int v=0;
    int d=999;
    for (int i=0;i<g->vertices;i++)
    {
        if (g->vertex[i].known==0 && g->vertex[i].dist<=d)
        {
            d=g->vertex[i].dist;
            v=i;
        }
    }
    return v;
}

void proper(struct graph *g)
{
    for (int i=0;i<g->vertices;i++)
    {
        printf("%d  %d  %d  %d ",i,g->vertex[i].known,g->vertex[i].dist,g-
>vertex[i].path);
        printf("\n");
    }
}

void dja(struct graph *g)
{
    int k=0;
    while (true)
    {
        k++;
        int n=numofunknown(g);
        if (n==0)
            break;
```

```
        else
        {
        int v=smallestvertex(g);
        g->vertex[v].known=1;
        for(int i=0;i<g->vertices;i++)
        {
            if (g->arr[v][i]>0)
            {
                if (g->vertex[i].known==0)
                {
                    int e=g->arr[v][i];
                    printf("e : %d\n",e);
                    if (g->vertex[v].dist+e<g->vertex[i].dist)
                    {
                        g->vertex[i].dist=g->vertex[v].dist+e;
                        g->vertex[i].path=v;
                    }
                }
            }
        }
        proper(g);
        printf("\n");
        }
    }
}

void pp(int s,struct graph *g)
{
    if (g->vertex[s].path!=-1)
    {
        pp(g->vertex[s].path,g);
        printf("--->");
    }
   printf("%d",s+1);
}
```

Dj.c

```
#include <stdio.h>
#include <stdlib.h>
#include "djadt.h"

void main()
{
```

NAME: S.KARTHIKEYAN 3122 22 5001 056
CSE B

```
    struct graph *g;
    g=(struct graph *)malloc(sizeof(struct graph));
    printf("enter the number of vertices:");
    int n;
    scanf("%d",&n);
    printf("enter the number of edges:");
    int e;
    scanf("%d",&e);
    init(g,n,e);
    struct edgepair *err[e];
    for (int i=0;i<e;i++)
    {
        err[i]=(struct edgepair*)malloc(sizeof(struct edgepair));
        printf("EDGE PAIR %d\n",(i+1));
        printf("enter the start ");
        int s;
        scanf("%d",&s);
        err[i]->start=s;
        printf("enter the end ");
        int en;
        scanf("%d",&en);
        printf("enter the weight : ");int we;scanf("%d",&we);
        err[i]->weight=we;
        err[i]->end=en;

    }
    create(g,err);
    display(g);
    printf("DIJKSTRA ALGOOO\n");
    inti(3,g);
    proper(g);
    dja(g);
    //display(g);
    printf("\n\n\n");
    int n1;
    printf("enter the vertex to find the shortest path :");
    scanf("%d",&n1);
    pp(n1-1,g);


}
```

NAME: S.KARTHIKEYAN 3122 22 5001 056

CSE B

OUTPUT:

```
enter the number of vertices:3
enter the number of edges:3
EDGE PAIR 1
enter the start 1
enter the end 2
enter the weight : 4
EDGE PAIR 2
enter the start 3
enter the end 1
enter the weight : 3
EDGE PAIR 3
enter the start 3
enter the end 2
enter the weight : 9
done create
0 4 0
0 0 0
3 9 0
```

```
DIJKSTRA ALGOOO
0  0  999  -1
1  0  999  -1
2  0  0  -1
e : 3
e : 9
0  0  3  2
1  0  9  2
2  1  0  -1

e : 4
0  1  3  2
1  0  7  0
2  1  0  -1

0  1  3  2
1  1  7  0
2  1  0  -1



enter the vertex to find the shortest path :2
3--->1--->2
```

NAME: S.KARTHIKEYAN **3122 22 5001 056**
CSE B

LEARNING OUTCOME:

Thus the dijkstra's algorithm is learnt, understood and implemented.